

API Security Risk Analysis Report

1. Introduction

This report presents a **read-only API Security Risk Analysis** conducted on the public demo API:

[JSONPlaceholder API](#)

The testing was performed using **Postman** to inspect:

- API endpoints
- Request/response behavior
- Headers
- Authentication mechanisms
- Authorization controls
- Data exposure risks

The assessment follows ethical testing guidelines and focuses only on publicly accessible demo endpoints.

2. Objective of the Assessment

The purpose of this assessment was to:

- Understand how APIs expose data
- Identify common API security weaknesses
- Evaluate authentication and authorization mechanisms
- Analyze response headers and exposed information
- Provide remediation recommendations similar to a professional AppSec review

3. Tools Used

Tool	Purpose
Postman	API testing and request inspection
Browser DevTools	Header and response inspection

Tool	Purpose
JSONPlaceholder	Public test API

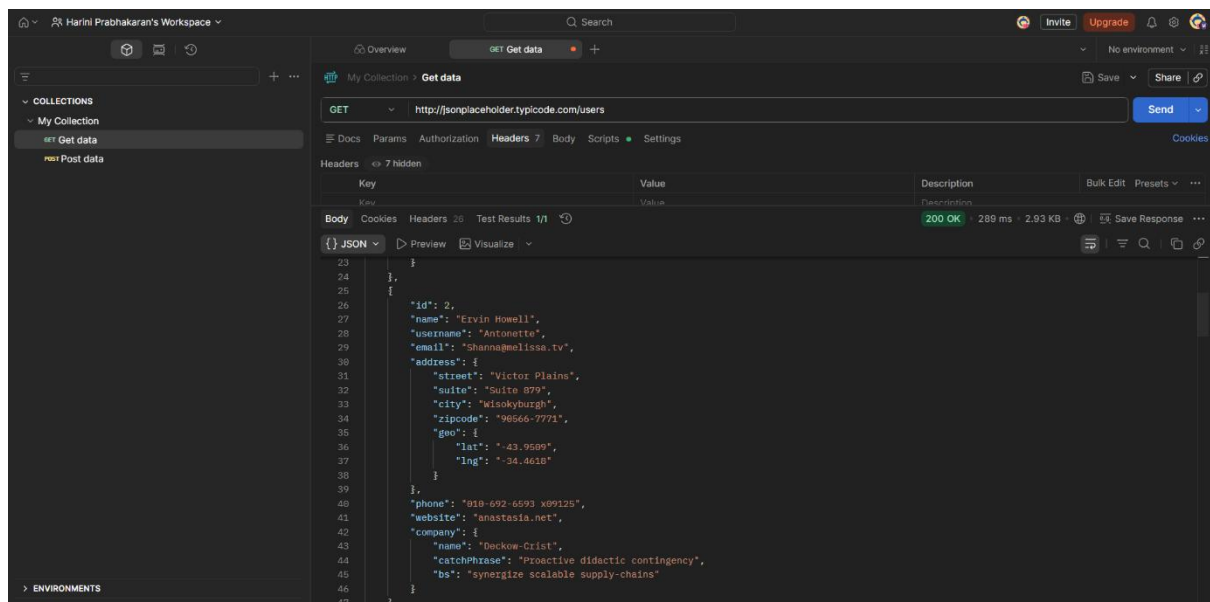
4. Scope of Testing

The following API operations were reviewed:

HTTP Method	Endpoint
GET	/users
GET	/users/2/posts
DELETE	/users/6
Header Inspection	Response metadata analysis
POST	/users

API Testing Evidence

5.1 GET Request – Retrieve Users



The API endpoint below was tested:

GET `http://jsonplaceholder.typicode.com/users`

Observed in Postman:

- The endpoint returned full user records
- No authentication token was required
- Sensitive-style information such as:
 - Username
 - email
 - phone number
 - address
 - company data was exposed publicly

This demonstrates a common API risk known as **Excessive Data Exposure**.

Security Observation

- Endpoint accessible without login
- No API key required
- No OAuth or JWT authentication
- Full user profile information visible

Risk Level

- Medium

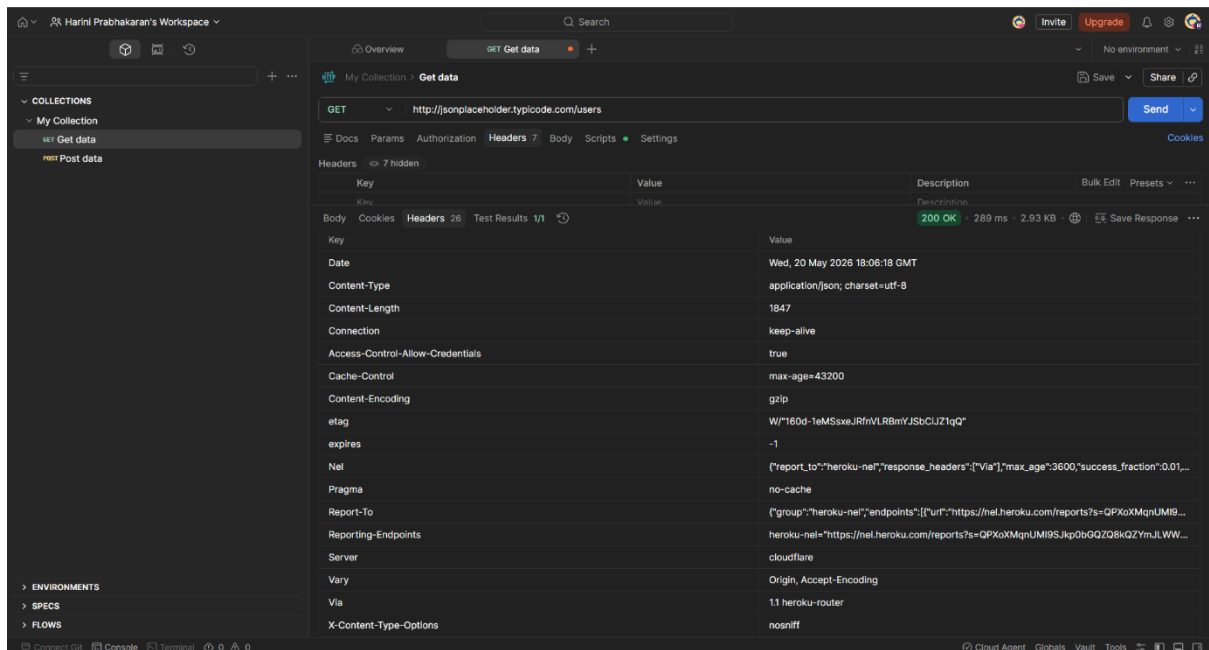
Business Impact

Attackers could scrape user information for:

- phishing
- spam campaigns
- social engineering
- reconnaissance activities

5.2 Response Header Inspection

The response headers were analyzed in Postman.



Observed headers included:

Content-Type: application/json Cache-Control: max-age=43200

Server: cloudflare

Security Findings

Positive Observations

- Content-Type correctly defined as JSON
- X-Content-Type-Options: nosniff present
- Compression enabled using gzip

Potential Concerns

- Server technology disclosure visible
- Cache duration relatively long
- No visible rate-limiting headers

Risk Level

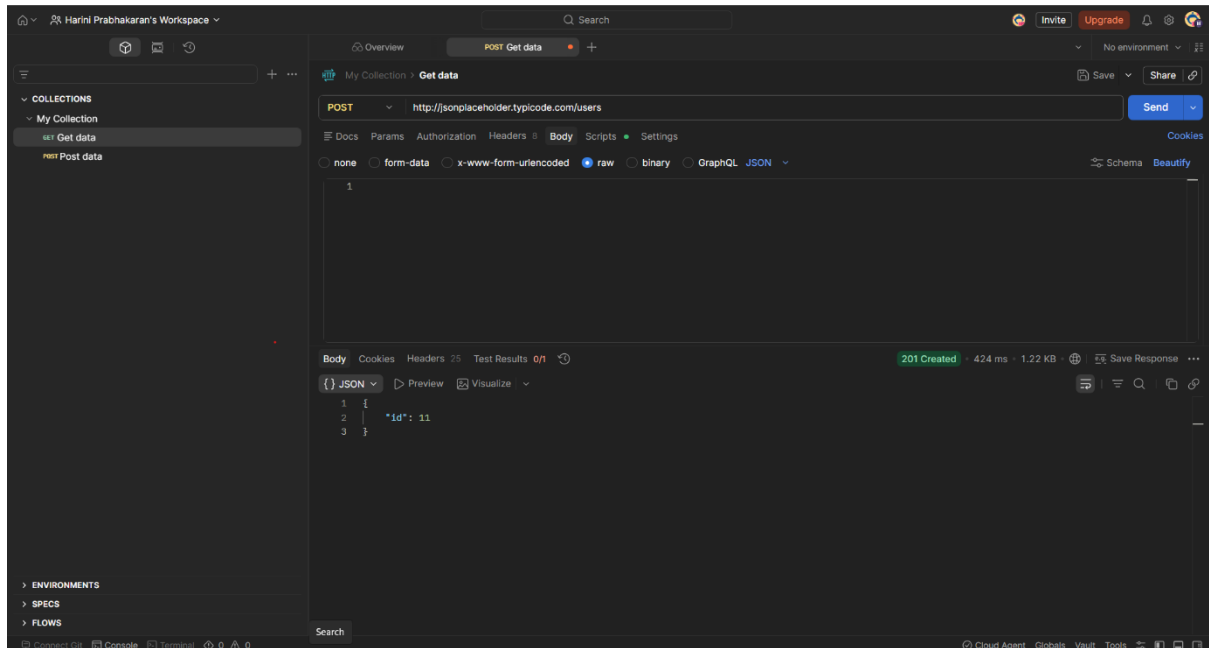
- Low to Medium

Business Impact

- Exposed infrastructure details may help attackers fingerprint the backend environment
- Lack of visible rate limiting can increase abuse risk.

5.3 POST Request Analysis

A POST request was sent successfully, and the API returned:



The screenshot shows:

- POST request execution
- Raw JSON body usage
- Successful resource creation (201 Created)

Security Analysis

- **Risk Identified: Weak Authentication Controls**

The API allows POST requests without authentication.

In real-world production APIs, this could allow attackers to:

- create unauthorized resources
- spam databases
- manipulate application data

Risk Level

- High

Business Impact

If implemented in production systems:

- attackers may create fake accounts

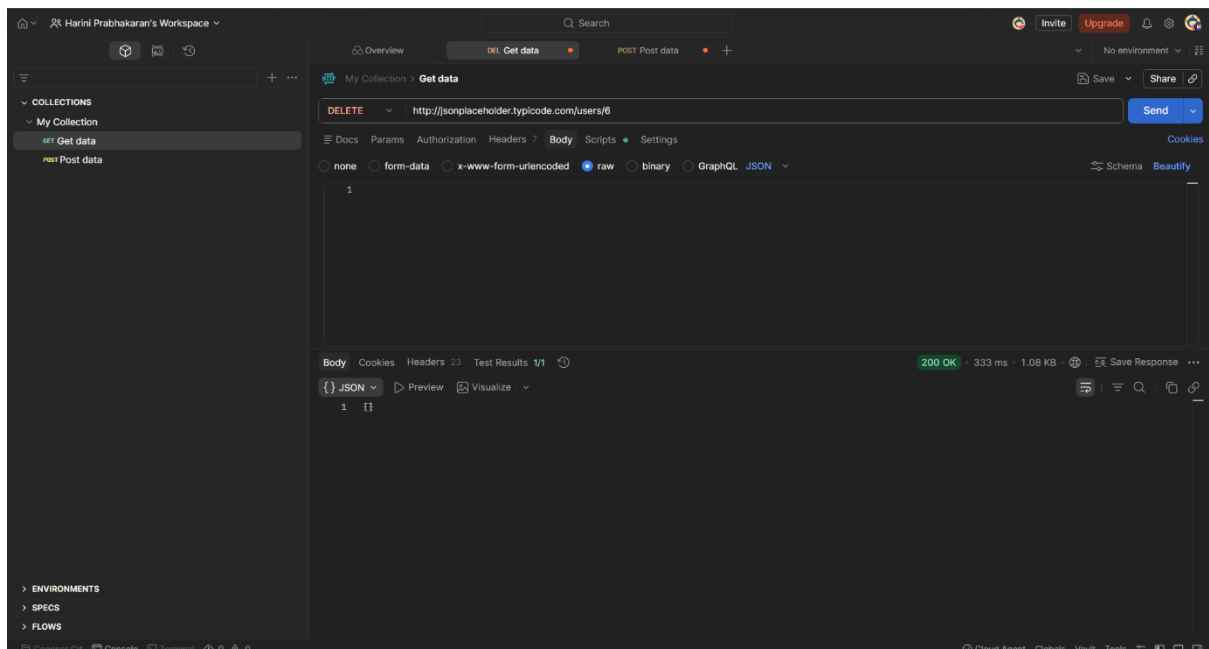
- malicious data may be injected
- application integrity may be compromised

Recommended Remediation

- Require authentication tokens
- Validate user identity before allowing POST actions
- Implement authorization checks
- Apply request validation and sanitization

5.3 DELETE Request Analysis

The following endpoint was tested:



Security Observation

- The endpoint accepted a DELETE request without authentication.

Although JSONPlaceholder is a demo API and does not actually delete data, this behavior highlights a critical real-world API risk.

Risk Identified

- Broken Authentication / Missing Authorization

Risk Level

- High

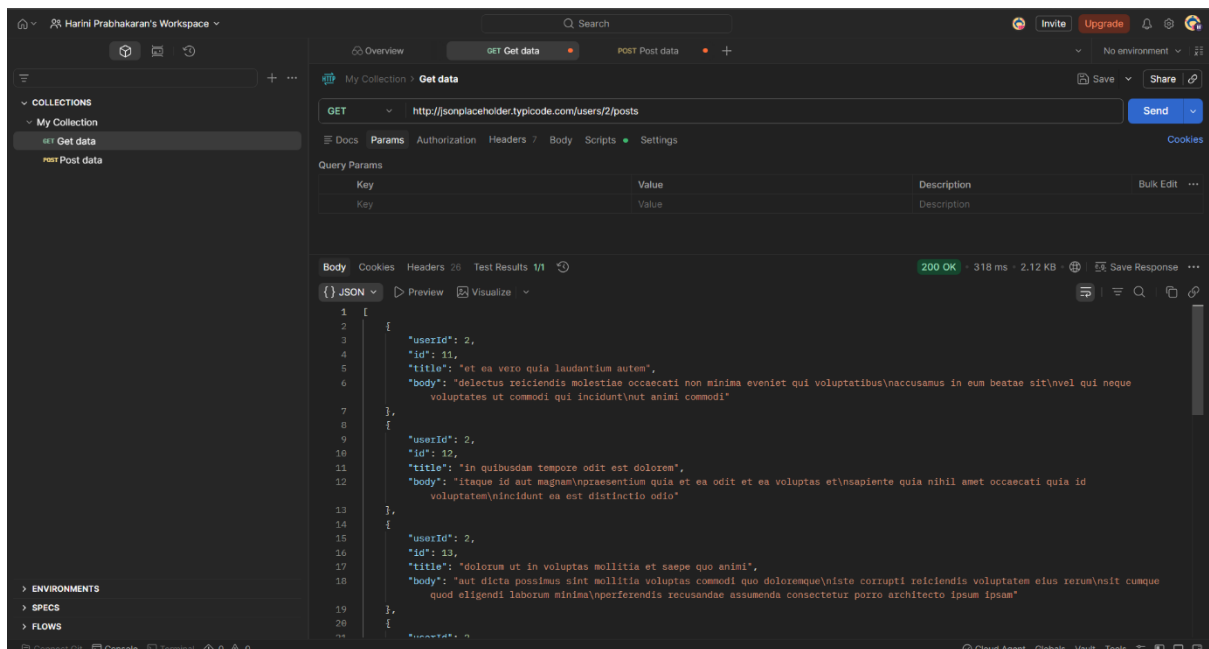
Business Impact

In production systems, unauthenticated DELETE operations could allow attackers to:

- remove customer data
- disrupt services
- manipulate records
- cause operational downtime

5.4 User-Specific Data Access- Fetch User Posts

The following endpoint was tested:



- GET <http://jsonplaceholder.typicode.com/users/2/posts>
- The API successfully returned posts associated with user ID 2.

Security Observation

- The endpoint allowed direct access to another user's content without verifying ownership.

Risk Identified

- Broken Object Level Authorization (BOLA)

This is one of the most common and dangerous API vulnerabilities.

Risk Level

- High

Business Impact

If implemented in real SaaS platforms, attackers could:

- access private customer records
- retrieve confidential data
- enumerate user accounts
- violate privacy regulations

Recommended Remediation

- Implement authentication mechanisms
- Use API keys or OAuth 2.0
- Restrict public access to sensitive endpoints

6.Key Security Risks Identified

Risk	Severity	Description
Unauthenticated Endpoints	High	Sensitive operations accessible publicly
Excessive Data Exposure	Medium	Full user details exposed unnecessarily
Broken Object Level Authorization	High	User data accessible without ownership validation
Missing Rate Limiting	Medium	Potential for API abuse or scraping
Server Information Disclosure	Low	Backend details visible in headers

7. Conclusion

The assessment demonstrates several common API security weaknesses frequently observed in real-world applications.

Using Postman, the API endpoints were inspected for:

- authentication
- authorization
- headers
- data exposure
- request handling

Although the tested platform is intentionally public and designed for learning purposes, the behaviors identified mirror serious vulnerabilities often discovered during professional API security audits.

The most significant risks identified were:

- Broken Object Level Authorization (BOLA)
- Missing Authentication Controls
- Excessive Data Exposure

Organizations should implement strong authentication, authorization, and rate-limiting controls to reduce API attack surfaces and protect sensitive business data.

- Harini Prabhakaran